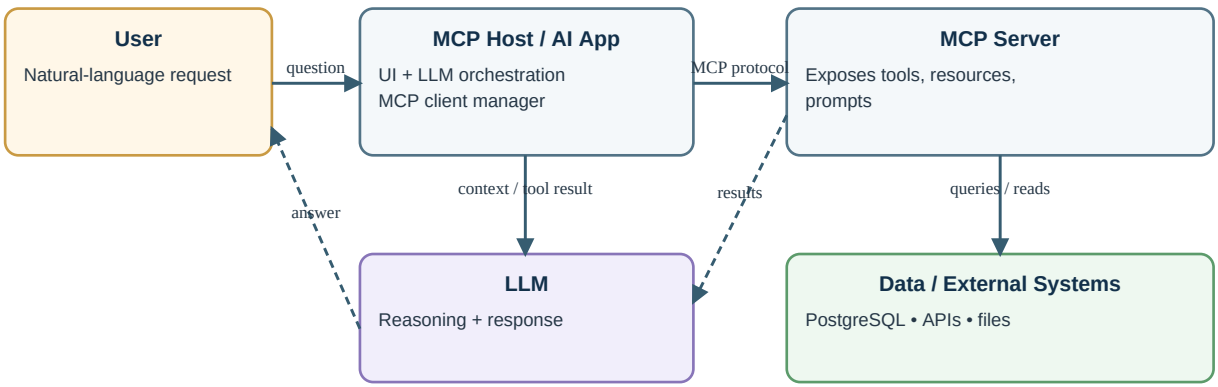
MODEL CONTEXT PROTOCOL (MCP)

Complete Notes: Basic to Advanced

Concepts • Architecture • UML diagrams • Python examples • Security • Deployment • Interview prep



MCP standardizes the connection; it does not define the LLM's reasoning strategy.

Practical learning guide using a Weather & Disaster Intelligence MCP server

Prepared: October 2026

How to use these notes

Read Parts 1–5 in order if you are new to MCP. Use Parts 6–10 when hardening a real server or preparing it for deployment. The examples use a Python MCP server that exposes weather forecast and disaster-alert data from PostgreSQL.

Version note: MCP and its SDKs evolve. Protocol capabilities, lifecycle methods, and SDK decorators can differ by specification and SDK release. Verify exact APIs against the version installed in your environment. The conceptual patterns in this guide remain useful across versions.

Contents

- 1. MCP fundamentals and what problem it solves
- 2. Architecture: host, client, server, LLM, data layer
- 3. Core primitives: tools, resources, prompts
- 4. Protocol messages, lifecycle, discovery, capabilities
- 5. Transports: stdio and Streamable HTTP
- 6. Build a Python MCP server for weather data
- 7. Agentic workflows and tool selection
- 8. Validation, errors, security, and trust boundaries
- 9. Advanced capabilities and production operations
- 10. Testing, debugging, deployment, and CI/CD
- 11. Weather MCP project architecture and roadmap
- 12. Interview questions, glossary, and official references

1. MCP fundamentals

1.1 What is MCP?

The Model Context Protocol (MCP) is an open protocol that standardizes how an AI application connects to external capabilities and context. An MCP server can expose tools to execute operations, resources to provide data, and prompts to provide reusable instruction templates.

MCP is often compared to a common connector standard: instead of writing a different integration for every AI application, a developer implements an MCP server and compatible hosts can discover and use its capabilities.

1.2 What MCP does—and does not do

MCP does	MCP does not automatically do
Standardizes discovery and invocation of capabilities.	Guarantee that an LLM reasons correctly.
Defines protocol messages and transport behavior.	Replace the LLM, agent framework, or business logic.
Expose tools, resources, prompts, and supported protocol features.	Make a database or API secure without application controls.
Help different clients integrate with the same server.	Predict weather or create a machine-learning model by itself.

1.3 Why it matters

• Reusability: one weather server can be connected to multiple compatible AI clients.

• Separation of concerns: the server owns data access; the host owns user interaction and LLM orchestration.

• Discoverability: clients can list available tools, resources, and prompts rather than relying only on undocumented function names.

• Governance: access controls, user confirmation, logging, and limits can be implemented around capabilities.

1.4 MCP vs direct function calling vs REST

Approach	Main purpose	Typical use
Direct function/tool calling	LLM provider-specific mechanism for selecting application functions.	Single application with functions defined in code.
REST API	HTTP resource and operation interface for software clients.	Backend/frontend integration, public APIs.
MCP	Standard protocol for AI applications to discover and use context/capabilities.	Share tools and context across MCP-compatible hosts.

MCP can call an HTTP API or database behind the scenes. It complements APIs; it does not necessarily replace them.

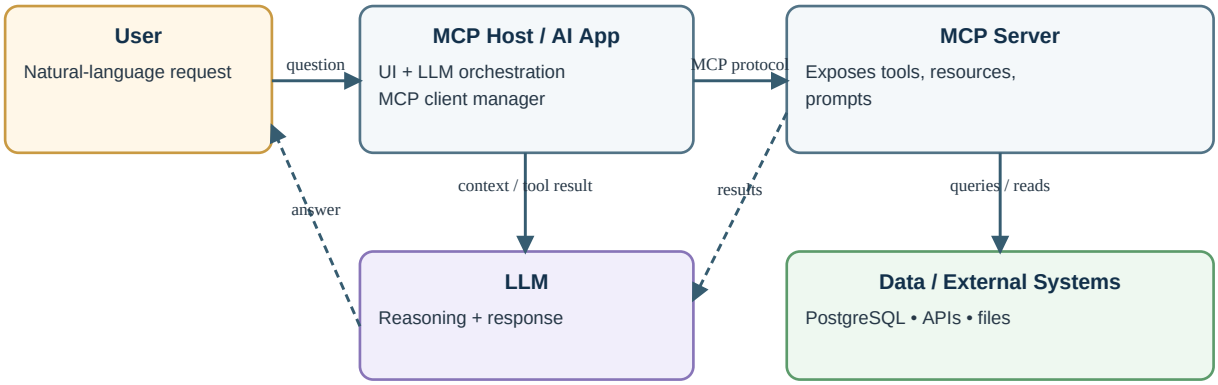
1.5 Terms to remember

Term	Meaning
Host	The AI application that coordinates the model, user experience, and MCP connections.
Client	Protocol component managed by the host; connects to one MCP server.
Server	Program exposing capabilities such as tools, resources, and prompts.
LLM	Model that interprets the request and generates a response or tool-call proposal.
Tool	Callable operation with a name, description, and input schema.
Resource	Addressable context/data, commonly identified by a URI.
Prompt	Reusable template that can be parameterized and supplied to the model.
Transport	Mechanism carrying protocol messages, such as stdio or Streamable HTTP.

2. Architecture and UML

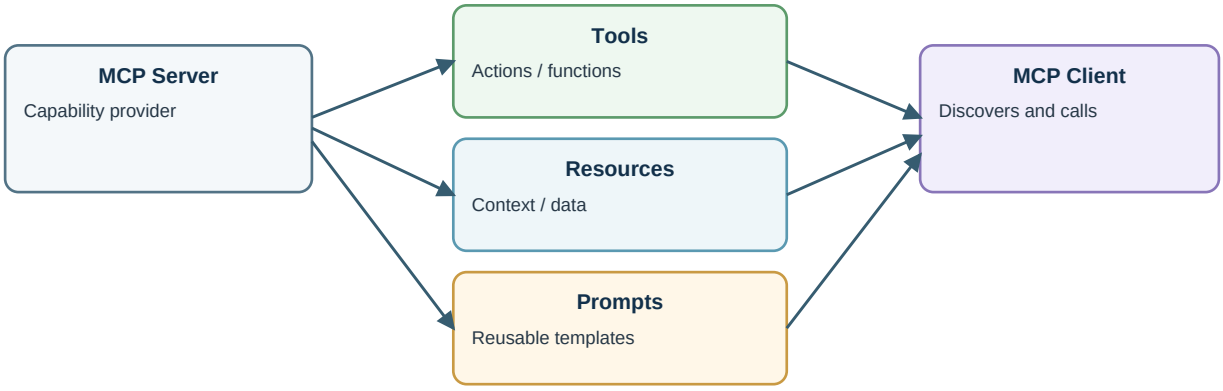
2.1 High-level component diagram

The host is responsible for integrating the LLM and one or more MCP clients. Each client connects to a specific server. A server can query a database, call an external API, or read files. The server normally does not need access to the complete conversation.



MCP standardizes the connection; it does not define the LLM's reasoning strategy.

2.2 MCP primitives at a glance

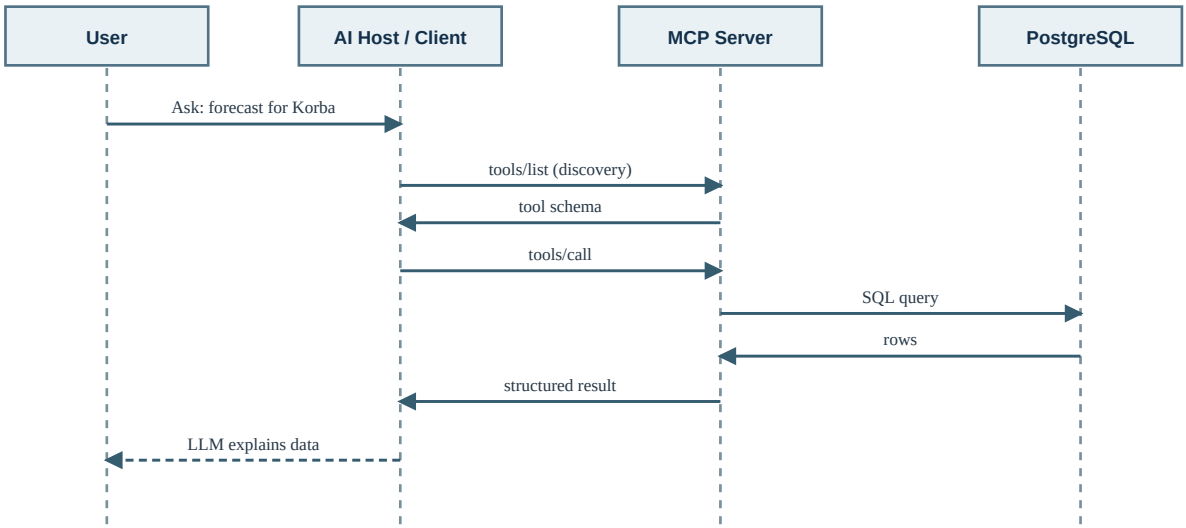


2.3 Responsibilities by layer

Layer	Responsibilities	Weather project example
Presentation / host	Chat UI, conversation state, consent, LLM orchestration.	User asks: “Forecast for Korba?”
MCP client	Connects to server; discovers and invokes capabilities.	VS Code AI client connects to server.py.
MCP server	Validates input and implements capability handlers.	get_district_forecast(district, limit).
Data access	SQL, connection pooling, API calls, data mapping.	Parameterized query against forecast table.
Data source	Stores or supplies facts.	PostgreSQL forecast and NDMA tables.

2.4 Sequence diagram: forecast request

This diagram illustrates a typical tool-driven request. Exact discovery and message sequences depend on the client and protocol version.



2.5 Important architectural distinction

The user's sentence is not converted into an MCP Prompt by the LLM. The host/LLM interprets the question, and the host may route a tool call through the MCP client. A prompt is a separate reusable template that the client can retrieve when appropriate.

3. The three core server primitives

3.1 Tools — actions

Tools are named operations that a client can discover and call. They are often model-controlled: the model proposes a tool and arguments, while the host/client decides whether and how to execute the call according to its policies.

• Use tools for calculations, database queries, API requests, searches, and controlled actions.

• Give each tool a precise name, description, input schema, and predictable output.

• Validate every argument in server code; model-generated arguments are not automatically trustworthy.

• Use read-only behavior by default. Require confirmation for consequential or irreversible actions.

Weather example

```
@mcp.tool()
def get_district_forecast(district: str, limit: int = 7):
    """Return available forecast rows for a district."""
    # Validate district and limit.
    # Execute parameterized SQL.
    # Return structured results.
```

3.2 Resources — context/data

Resources expose data that a client can read and provide as context. They are commonly identified by URIs. Examples include a database catalog, schema documentation, policy documents, or a report. Resources are not the same as executable operations.

• Use a stable URI scheme, e.g. weather://database/catalog.

• Describe the resource clearly and return the declared content type.

• Avoid returning huge datasets; use bounded resources or tools for filtered queries.

• Treat resource content as data, not as trusted instructions.

Weather catalog example

```
@mcp.resource(
    "weather://database/catalog",
    name="Weather Database Catalog",
    description="Describes forecast and disaster-alert datasets.",
    mime_type="application/json",
)
def get_weather_database_catalog() -> str:
    return json.dumps({
        "datasets": [
            "weatherdata.district_wise_7dayfc_severity",
            "weatherdata.disaster_ndma"
        ],
        "limitations": [
            "Forecast availability is limited to stored forecast dates."
        ]
    })
```

3.3 Prompts — reusable instruction templates

Prompts are templates a client can retrieve and fill with arguments. They help standardize repeated workflows such as risk analysis, incident triage, or code review. A prompt returns instructions or messages; it does not itself execute a database query.

```
@mcp.prompt()
def analyze_weather_risk(
    district: str,
    focus: str = "forecast and active disaster alerts"
) -> str:
    """Analyze a district's weather forecast and risks."""
    return f"""
Analyze {district}.
Focus: {focus}.
Use available forecast and alert tools.
```

```
Separate retrieved facts from interpretation.
State missing data and forecast limitations.
Do not invent weather values.
"""
```

The decorator syntax above is illustrative for SDKs that support this API. Confirm the exact signature and prompt-return format for the installed Python SDK.

3.4 Choosing the right primitive

Question	Best fit	Why
“Fetch the latest forecast for Korba.”	Tool	Requires a query or operation.
“What tables and fields are available?”	Resource	Provides reference context.
“Follow our standard risk-analysis procedure.”	Prompt	Reusable instructions for the model.
“Delete an alert record.”	Tool + strong authorization/confirmation	Side effect requiring safeguards.

3.5 Discovery methods (conceptual)

• tools/list → discover tool names, descriptions, and input schemas.

• tools/call → invoke a tool with arguments.

• resources/list and resources/read → discover and read resources.

• prompts/list and prompts/get → discover and instantiate prompt templates.

Method names and lifecycle details are versioned by the specification. Use the protocol and SDK documentation matching your client/server release.

4. Protocol foundations

4.1 JSON-RPC 2.0

MCP's data layer uses JSON-RPC-style messages. A request has a method and parameters; a response correlates to the request ID. Notifications do not expect a response. The transport carries these messages between client and server.

```
{
  "jsonrpc": "2.0",
  "id": 17,
  "method": "tools/call",
  "params": {
    "name": "get_district_forecast",
    "arguments": {"district": "Korba", "limit": 7}
  }
}
```

A real protocol message may include required metadata and fields depending on the specification version. Treat this snippet as a teaching example, not a complete wire-level conformance fixture.

4.2 Requests, responses, and notifications

Message type	Purpose	Example
Request	Ask the other side to perform an operation.	tools/call
Response	Return result or error for a request ID.	Forecast rows or error details.
Notification	Inform the other side without requiring a response.	Capability/list changed notification where supported.

4.3 Lifecycle and capability discovery

A client and server must agree on protocol compatibility and the capabilities they support. MCP versions have changed over time; some releases use an initialization handshake, while newer specification revisions may use different discovery conventions. Do not hard-code assumptions from a different version into your application.

- Identify the exact MCP specification version and SDK version.
- Discover supported capabilities before relying on optional features.
- Handle unsupported methods or protocol-version mismatches gracefully.
- Keep the server's advertised capabilities consistent with what it actually implements.

4.4 Schemas and structured results

Input schemas help clients understand required and optional arguments. Structured output, where supported by the SDK/protocol version, improves reliability by letting clients validate result shape. Even with schemas, server-side validation remains mandatory.

4.5 Errors

- Protocol errors: malformed message, unknown method, unsupported capability.
- Application errors: unknown district, database unavailable, invalid date range.
- Operational errors: timeout, rate limit, network failure.
- Return safe, actionable error messages; log detailed diagnostics server-side without leaking secrets.

5. Transports: stdio and Streamable HTTP

5.1 stdio

The host launches a local MCP server process and communicates through standard input and output. This is convenient for desktop AI tools, local development, and servers that run beside the client.

• Keep stdout reserved for protocol messages; use stderr for diagnostics.

• Do not print banners, debug output, or SQL logs to stdout in a stdio server.

• Use absolute executable paths and a controlled working directory in client configuration.

• Protect environment variables and local files accessible to the process.

5.2 Streamable HTTP

Streamable HTTP is designed for network-accessible MCP servers. It uses HTTP-based message exchange and can support streaming behavior where applicable. It requires production-grade network security, authentication, authorization, and operational monitoring.

Topic	stdio	Streamable HTTP
Typical deployment	Local process launched by host.	Remote server reachable over a network.
Communication	stdin/stdout streams.	HTTP requests; streaming support as defined by protocol.
Authentication	Usually relies on local process/config trust boundaries.	Needs explicit auth and access control for protected endpoints.
Operational scaling	Usually per host/client process.	Can serve multiple clients, depending on implementation.
Common use	VS Code local MCP server.	Shared team or cloud MCP service.

5.3 Choosing a transport

For your current learning project, stdio is appropriate while using MCP Inspector and VS Code. When you need a centrally hosted service for multiple users, evaluate Streamable HTTP and add authentication, authorization, TLS, rate limiting, and deployment observability.

5.4 Configuration checklist

• Use the intended Python interpreter from the project virtual environment.

• Set the correct working directory and environment-file path.

• Do not commit .env files or tokens to Git.

• Test the server directly and then through MCP Inspector/client.

• Restart the host after configuration changes when required.

6. Build a Python Weather MCP server

6.1 Project structure

```
Weather MCP/
■■■ server.py
■■■ db.py                # connection and query helpers
■■■ services/
■   ■■■ forecast_service.py
■   ■■■ alert_service.py
■■■ tests/
■   ■■■ test_validation.py
■   ■■■ test_queries.py
■■■ .env                 # local secrets; never commit
■■■ .gitignore
■■■ requirements.txt
```

6.2 Environment setup

```
py -3.11 -m venv py-env
py-env\Scripts\activate
python -m pip install --upgrade pip
pip install mcp psycopg[binary] python-dotenv
pip freeze > requirements.txt
```

Package names and available classes differ across MCP Python SDK generations. Your existing project uses an `MCPServer` API; many current tutorials use `FastMCP`. Avoid mixing snippets from these APIs without checking your installed package.

6.3 Configuration

```
# .env (example names only; use your real values locally)
DB_HOST=localhost
DB_PORT=5432
DB_NAME=weather_db
DB_USER=weather_readonly
DB_PASSWORD=replace_with_secret

# .gitignore
.env
py-env/
__pycache__/
*.pyc
.pytest_cache/
```

6.4 Database query principles

Use parameterized SQL, a least-privilege database role, bounded result sizes, timeouts, and explicit handling for missing rows. Never build SQL by concatenating user-provided city or district names into the query string.

```
# psycopg pattern (illustrative)
query = '''
    SELECT days, date, district, temp_min, temp_max,
           rain_percent, wind, humidity, condition_text
    FROM weatherdata.district_wise_7dayfc_severity
    WHERE district ILIKE %s
    ORDER BY date
    LIMIT %s
'''
cur.execute(query, (district, limit))
rows = cur.fetchall()
```

6.5 Validate inputs before querying

```
def validate_district_and_limit(district: str, limit: int) -> tuple[str, int]:
    district = district.strip()
    if not district:
        raise ValueError("district must not be empty")
    if len(district) > 100:
        raise ValueError("district is too long")
    if limit < 1 or limit > 31:
        raise ValueError("limit must be between 1 and 31")
    return district, limit
```

6.6 Example tool contracts

Tool	Inputs	Expected behavior
get_district_forecast	district, limit	Returns stored forecast rows for a district.
get_high_risk_districts	limit	Returns districts ranked by configured severity rules.
get_disaster_alerts	disaster_type, district, state_ut, active_only, limit	Filters normalized NDMA alerts.
get_district_disaster_summary	district	Combines available forecast and alert data.

6.7 What a good result should look like

Prefer consistent, explicit result fields. Include dates, units, source timestamps, and data availability indicators when possible. Do not return only a vague string if downstream logic needs to reason over values.

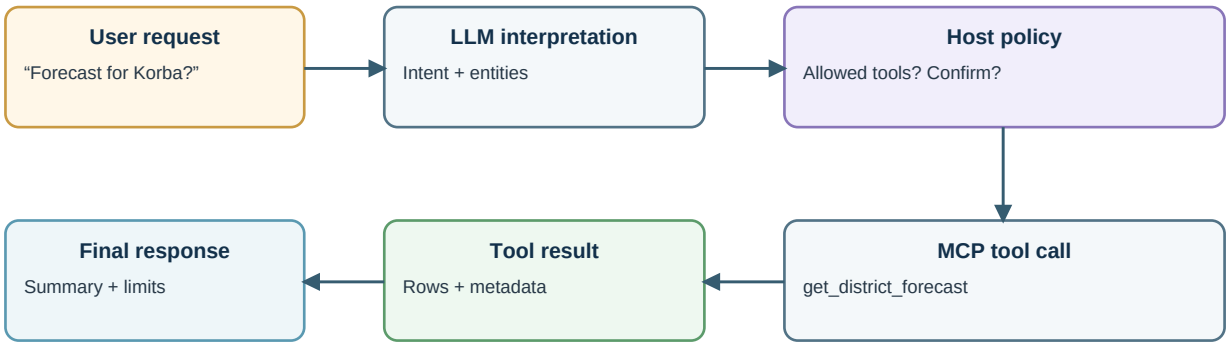
```
{
  "district": "Korba",
  "forecast_period": {
    "start_date": "YYYY-MM-DD",
    "end_date": "YYYY-MM-DD"
  },
  "source": "weatherdata.district_wise_7dayfc_severity",
  "rows": [],
  "limitations": [
    "Only dates available in the stored forecast are returned."
  ]
}
```

7. Agentic AI and MCP orchestration

7.1 What makes a workflow agentic?

An agentic workflow uses a model to interpret goals, select tools, inspect results, and decide the next step within defined limits. MCP provides the standardized connection to capabilities; the host or agent framework manages the reasoning loop.

7.2 Tool-selection flow



7.3 Typical tool loop

1. The user submits a question.
2. The host sends the request and relevant conversation context to the LLM.
3. The LLM proposes a tool and arguments, if a tool is needed.
4. The host/client checks tool availability and applicable policy or confirmation requirements.
5. The MCP client sends the tool call to the MCP server.
6. The server validates arguments and performs the operation.
7. The result returns to the host, which supplies it to the LLM.
8. The LLM synthesizes an answer grounded in the returned evidence.

7.4 Example: “Is Korba at risk today?”

• Fetch the available forecast for Korba.

• Fetch active alerts filtered by district and effective time.

• Check timestamps and whether the alert is active for the requested date.

• Summarize forecast conditions and alerts separately.

• Explain risk using known severity rules; do not invent an official risk score.

• State missing data and source freshness.

7.5 Why an MCP Prompt is not the same as a tool call

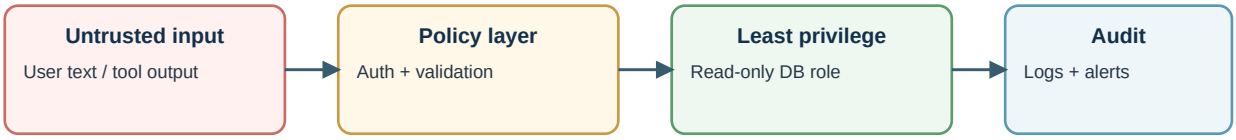
A prompt is a reusable set of instructions that a client can retrieve. A tool is an operation that executes. The LLM does not have to retrieve an MCP prompt for every question; a host can directly expose tools to the model. For complex, repeatable tasks, a prompt can instruct the model to use a sequence of tools.

7.6 Guardrails against hallucination

- Require evidence for numeric claims: use returned tool values, not model memory.
- Show dates and units; distinguish observed, forecast, and estimated data.
- If no matching district is found, ask for clarification or report no data.
- Never claim predictions beyond the provider's forecast horizon without a validated source/model.
- Use deterministic code for severity ranking and business rules wherever possible.

8. Validation, errors, and security

8.1 Threat model



Never treat retrieved content as trusted instructions.

Validate arguments • limit rows • parameterize SQL • protect credentials • confirm side effects

8.2 Input validation

• Validate types, lengths, allowed ranges, enum values, and date formats.

• Bound `limit`, pagination, time windows, and response size.

• Normalize district names carefully; avoid ambiguous matching that silently returns the wrong region.

• Reject unknown filters rather than quietly ignoring them.

8.3 SQL and database safety

• Use parameterized queries; never interpolate user text into SQL syntax.

• Use a dedicated read-only database account for query-only tools.

• Grant access only to required schemas/tables and columns.

• Set statement timeouts and connection timeouts.

• Use transactions appropriately and close cursors/connections reliably.

• Avoid returning internal connection strings, stack traces, or sensitive rows.

8.4 Prompt injection and untrusted content

Tool results, database fields, web pages, alert messages, and resource contents may contain malicious or misleading instructions. Treat them as untrusted data. They must not override system policy or grant permission to invoke unrelated tools.

• Keep instructions and retrieved data clearly separated in the host/agent design.

• Use allowlists for tools and constrain arguments to the user's intended task.

• Do not let tool output request secrets or change authorization rules.

• Use human confirmation for consequential actions.

• Audit tool calls and their outcomes.

8.5 Authentication and authorization

For remote deployments, authenticate clients and authorize each capability. Use TLS, secure token handling, least privilege, short-lived credentials where appropriate, and a documented token lifecycle. Follow the current MCP authorization specification and your chosen SDK's implementation guide.

8.6 Error handling pattern

```
try:
    district, limit = validate_district_and_limit(district, limit)
    rows = query_forecast(district, limit)
    return {
        "ok": True,
        "district": district,
        "count": len(rows),
        "rows": rows
    }
except ValueError as exc:
    # Safe, actionable validation error
    return {"ok": False, "error": str(exc)}
except DatabaseUnavailable:
    # Log detailed diagnostics server-side
    return {
        "ok": False,
        "error": "Forecast data is temporarily unavailable."
    }
```

Adapt the error shape to the MCP SDK and client conventions. Avoid swallowing every exception and returning an empty result, because that makes failures look like valid “no data” outcomes.

9. Advanced MCP concepts

9.1 Notifications and dynamic capability lists

Some protocol features let servers notify clients when supported lists change, such as when new tools become available. Clients should not assume that a capability list is immutable if the server advertises dynamic changes.

9.2 Pagination, progress, and cancellation

• Pagination: split large lists into bounded pages instead of returning unbounded datasets.

• Progress: communicate progress for operations that take time, where supported.

• Cancellation: allow clients to cancel work that is no longer needed.

• Timeouts: use explicit deadlines for database/API calls even if cancellation exists.

9.3 Elicitation / asking the user for more input

Some MCP versions and client capabilities support a server asking the client to collect additional user input. Use this only when the client supports the feature and the missing information is genuinely required. For example, a location-ambiguous query may need the state or country.

9.4 Sampling and version-sensitive features

Important version caveat: advanced client/server features evolve. Some features available in older protocol revisions may be deprecated or replaced in newer revisions. Before implementing sampling, roots, or experimental task extensions, check the exact protocol version and SDK support rather than copying an older tutorial.

9.5 Long-running tasks

For long-running work—large report generation, batch geospatial analysis, or model training—consider a job/task design with status, progress, cancellation, and result retrieval if the protocol version and client support it. Do not hold a synchronous tool call open indefinitely.

9.6 Composing multiple MCP servers

A host can connect to multiple servers, for example a weather server, a GIS server, and a document-search server. The host coordinates the model and keeps each server's connection and permission boundary separate.

9.7 Observability

• Log tool name, duration, success/failure, and a correlation ID.

• Avoid logging passwords, tokens, raw sensitive content, or full prompts unnecessarily.

• Measure database latency, API latency, timeouts, error rates, and result sizes.

• Use structured logs and tracing; consider OpenTelemetry for production.

• For stdio, send diagnostic logs to stderr, not stdout.

9.8 Caching

Cache only when the data's freshness and permissions permit it. Forecasts and active alerts are time-sensitive: include source timestamps, TTLs, and invalidation rules. Never reuse one user's authorization-scoped result for another user without a safe cache key.

9.9 Idempotency and side effects

A read-only forecast tool is naturally safer than a tool that changes records. For write actions, design idempotency, audit logging, confirmation, authorization, and recovery. Do not assume the model will avoid duplicate tool calls.

10. Testing, debugging, deployment

10.1 Testing layers

Test layer	What to test
Unit	Input validation, severity mapping, date parsing, response formatting.
Database integration	SQL correctness, empty results, permission failures, timeouts.
MCP protocol	Tools/resources/prompts discover correctly; arguments and outputs conform.
Host integration	Configured client starts server and invokes tools successfully.
Security	Invalid input, SQL injection attempts, unauthorized calls, oversized results.
End-to-end	User question leads to evidence-backed answer and appropriate limitations.

10.2 MCP Inspector workflow

1. Start the server with the same interpreter and environment as the AI host.
2. Open MCP Inspector and connect using the correct transport.
3. Inspect available tools and their schemas.
4. Call each tool with valid inputs and edge cases.
5. Read resources and verify MIME type/content.
6. Retrieve prompts and verify arguments render correctly.
7. Inspect server logs for errors; keep stdio protocol output clean.

10.3 Troubleshooting checklist

Symptom	Likely cause / next check
Server does not start	Wrong Python interpreter, import error, missing .env, invalid path.
Tool not visible	Decorator/registration issue, server not restarted, client discovery/cache.
Tool returns no rows	District spelling, date filter, timezone, source table freshness.
JSON/protocol parsing error	Debug text written to stdout in stdio mode.
Database authentication fails	Credentials, host/port, pg_hba.conf, role privileges, network rules.
Slow call	Missing index, unbounded result, slow upstream API, no timeout.
LLM invents values	No grounding instruction, poor result schema, missing date/source context.

10.4 Deployment checklist

- Pin dependency versions and record the supported MCP protocol/SDK version.
- Store secrets in environment variables or a secret manager; rotate credentials.
- Use a non-admin service account and least-privilege database role.
- Add health checks, logs, metrics, timeouts, and alerting.

- Secure remote transport with TLS and appropriate authentication.
- Set resource limits and rate limits; protect against denial-of-service patterns.
- Run automated tests in CI before deployment.
- Document how to roll back a broken release.

10.5 CI/CD outline

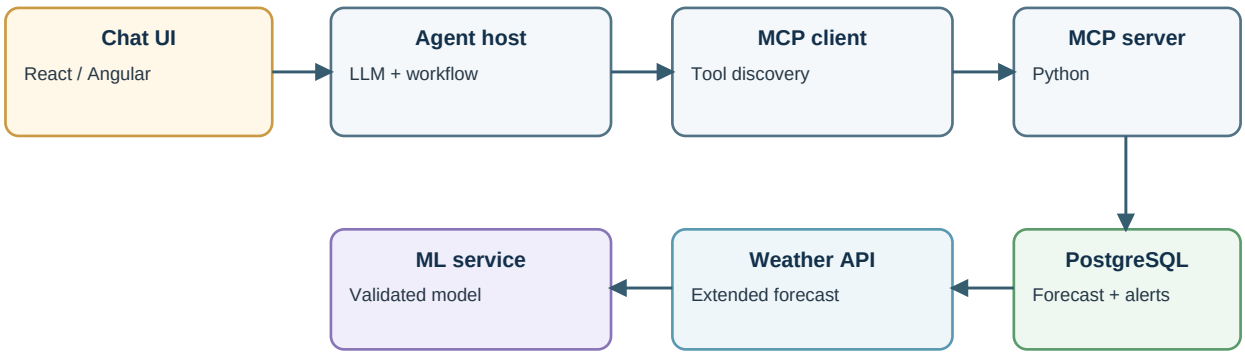
```
Developer push
    -> lint + type checks
    -> unit tests
    -> integration tests (test database)
    -> package / build
    -> security/dependency scan
    -> deploy to staging
    -> smoke test MCP capabilities
    -> approval / deploy production
    -> monitor latency, errors, and freshness
```

11. Your Weather & Disaster Intelligence MCP project

11.1 Current capabilities

Capability	Purpose
get_district_forecast	Retrieve stored district-wise 7-day forecast.
get_high_risk_districts	Identify districts with elevated severity according to configured fields.
get_disaster_alerts	Filter normalized NDMA alerts by type, district, state, and active status.
get_district_disaster_summary	Combine forecast and disaster-alert evidence for a district.
weather://database/catalog	Explain available datasets, fields, and forecast limitations.
analyze_weather_risk prompt	Reusable instructions for a structured risk analysis, if registered and supported.

11.2 Recommended end-to-end design



Keep forecast computation in a data/model service; use the LLM for orchestration and explanation.

11.3 Data freshness and forecast honesty

• Return the forecast's valid date/time and the source's issue/insert timestamp when available.

• Distinguish “current weather”, “provider forecast”, “stored forecast”, and “model estimate”.

• Do not describe a 7-day forecast table as an unlimited future forecast.

• For horizons beyond available data, call an appropriate forecast provider or a separately validated model.

• For historical trend analysis, verify that historical observations exist and have consistent units and quality controls.

11.4 A safe future-prediction roadmap

1. Add a tool that reports the available forecast horizon and last update time.

2. Integrate a documented external forecast provider for extended forecasts.

3. Build a historical-data quality pipeline: missing values, units, timestamps, station/district mapping.

4. Train and validate a baseline model using chronological train/validation splits.

5. Compare against naive and provider baselines; report metrics and uncertainty.

6. Expose model output through a separate MCP tool with model version, horizon, and confidence/uncertainty metadata.

7. Have the LLM explain the output without inventing precision or presenting an experimental estimate as an official warning.

11.5 Portfolio-quality improvements

• Add typed schemas and consistent JSON output contracts.

• Add tests for unknown districts, empty data, expired alerts, and database outages.

• Use read-only DB credentials and parameterized SQL.

• Add structured logging, latency metrics, and source freshness checks.

• Write architecture, sequence, and deployment diagrams in the repository README.

• Create a small evaluation set of realistic questions and score answer grounding/correctness.

12. Interview preparation

12.1 Common questions and concise answers

What is MCP?

An open protocol standardizing how AI hosts connect to servers that expose context and capabilities.

What are host, client, and server?

The host coordinates the AI application; the client maintains a protocol connection to one server; the server exposes capabilities.

What is the difference between a tool and a resource?

A tool executes an operation; a resource provides addressable context/data.

What is an MCP Prompt?

A reusable, parameterized instruction template that a client can retrieve and provide to a model.

Does MCP itself contain the LLM?

No. MCP defines integration and context exchange; the host decides how to use an LLM.

Does the LLM automatically turn a question into an MCP Prompt?

No. The LLM interprets the question and may propose a tool call. Prompts are separate templates retrieved by the client when useful.

Why use MCP instead of a direct integration?

A standard capability interface can be reused across compatible AI hosts and clients.

What transports should you know?

stdio for local process communication and Streamable HTTP for remote network access; confirm current spec and SDK support.

How do you secure a database MCP server?

Least-privilege database role, parameterized SQL, validation, bounded results, authentication where needed, timeouts, auditing, and secret management.

How do you prevent hallucinated weather data?

Ground answers in tool results, return dates/units/source metadata, validate output, and state when data is missing.

What happens if a tool fails?

Return a safe, structured failure or protocol error as appropriate, log diagnostics, and avoid presenting failure as valid empty data.

How would you test an MCP server?

Unit tests, database integration tests, Inspector capability tests, host integration tests, security tests, and end-to-end grounding evaluations.

12.2 Glossary

Term	Definition
Agent	Application workflow that can plan or choose actions using model output and tools.
Capability	A feature a client or server advertises/supports.
JSON-RPC	Message format for remote procedure-call style communication.
MCP host	AI application coordinating clients, context, and the model.
Prompt injection	Untrusted content attempting to manipulate model behavior or policies.
Schema	Formal definition of expected input or output fields and types.
stdio	Standard input/output streams used for local process communication.
Streamable HTTP	HTTP-based MCP transport for remote communication, with streaming behavior as specified.
TTL	Time-to-live; maximum intended cache age.
Tool grounding	Using actual tool results as evidence for an answer.

12.3 Suggested study plan

Stage	Study / deliverable
1. Fundamentals	Explain host/client/server and tools/resources/prompts without notes.
2. Build	Create one read-only tool and validate its arguments.
3. Context	Add a resource and a parameterized prompt; test both in Inspector.
4. Integration	Connect the server to an AI host and trace a tool call end-to-end.
5. Security	Add limits, timeouts, parameterized SQL, least-privilege credentials, and error handling.
6. Production	Add tests, logs, metrics, pinned dependencies, and deployment documentation.
7. Agentic workflow	Combine forecast and alerts; evaluate whether answers stay grounded.

13. Official references and version notes

Use the official specification and SDK documentation as the source of truth. The MCP protocol is versioned, and newer revisions can change lifecycle details or deprecate features. Always match examples to the SDK installed in your project.

MCP Architecture Overview: https://modelcontextprotocol.io/docs/learn/architecture

MCP Specification: https://modelcontextprotocol.io/specification/

MCP Tools: https://modelcontextprotocol.io/specification/2026-07-28/server/tools

MCP Python SDK: https://github.com/modelcontextprotocol/python-sdk

MCP Inspector: https://github.com/modelcontextprotocol/inspector

MCP Authorization: https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization

MCP Transports: https://modelcontextprotocol.io/specification/2026-07-28/basic/transports

Final checklist

• I can explain MCP host, client, and server.

• I know when to use tools, resources, and prompts.

• I can explain the tool-call sequence from user request to final answer.

• I can test the server with Inspector and a real AI host.

• I validate inputs and use parameterized SQL.

• I understand the security differences between stdio and remote HTTP deployment.

• I distinguish stored forecasts, provider forecasts, and model-generated estimates.

• I check official docs for protocol and SDK version-specific changes.